

Параллельные высокопроизводительные вычисления

Отчёт по заданию

**“многопоточная реализация операций с сеточными
данными на неструктурированной смешанной сетке,
решение СЛАУ”**

Вариант – Б1

Группа: 523

Повжик Юрий Максимович
23.11.2025

Содержание

2 Описание задания и программной реализации.	3
2.1 Краткое описание задания	3
2.2 Краткое описание программной реализации.....	5
3 Исследование производительности	6
3.1 Характеристики вычислительной системы	6
3.2 Результаты измерений производительности	7
3.2.1 Последовательная производительность	7
3.2.2 Параллельная производительность	9
4 Анализ полученных результатов	15

2 Описание задания и программной реализации.

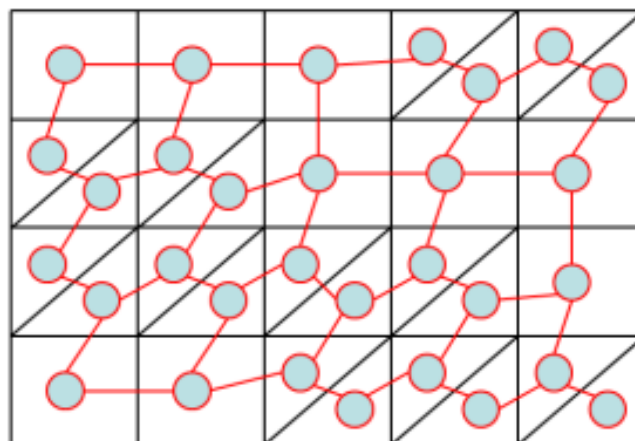
2.1 Краткое описание задания

Задание: многопоточная реализация операций с сеточными данными на неструктурированной смешанной сетке, решение СЛАУ.

Входными данными для задачи являются:

- N_x, N_y – число клеток в решетке по вертикали и горизонтали
- $K1, K2$ – параметры для количества однопалубных и двухпалубных кораблей треугольных и четырехугольных элементов
- $maxit$ - максимальное число итераций метода сопряженных градиентов с диагональным предобуславливателем
- eps - критерий остановки (ϵ), которым определяется точность решения
- Y - флаг для вывода отладочной печати

Согласно варианту задания был реализован следующий вариант интерпретации входных данных:



$N_x = 5$
 $N_y = 4$
 $K1 = 3$
 $K2 = 4$

Для решения задачи необходимо реализовать 3 функции:

- **Generate** – на вход принимает $N_x, N_y, K1, K2$.

Возвращает:

N – размер матрицы = число вершин в графе

IA, JA – портрет разреженной матрицы смежности графа, дополненный главной диагональю в формате ELLPACK

- Fill – принимает размер матрицы и ее портрет.

Возвращает:

A – массив ненулевых коэффициентов матрицы

b – вектор правой части

- Solve – принимает полученные A и b вместе с eps и maxit

Возвращает:

x – вектор решения

n – количество выполненных итераций

res – L2 норма невязки

2.2 Краткое описание программной реализации

Для проверки и дальнейшего хранения входных параметров задачи был реализован класс **StartData**, принимающий `argc` и `argv`.

В случае успешной проверки мы формируем стартовую матрицу, класс которой назван **StartMatrix**. Он принимает **StartData** и строит по полученным данным матрицу соответствующего вида. Элементы матрицы имеют тип **MatrixElem**, хранящий три значения: число элементов, номер верхнего и нижнего. Метод **fillMatrix** отвечает за заполнение начальной матрицы согласно заданным параметрам.

Построением формата ELLPACK занимается класс **ELLPACK**, принимающий на вход **StartMatrix**. Он хранит матрицу значений A , JA . Метод **fillJA** заполняет матрицу JA . Места отсутствующих элементов заполняем последним полученным индексом. Методы **addPair** и **addElem** вспомогательные и используются для добавления ребер между точками. Метод **fill** заполняет матрицу A сгенерированными значениями.

Функция **fillB** заполняет массив b .

Также в коде присутствуют реализации функции **spmv** для диагональной матрицы и матрицы в формате ELLPACK, **dotVec**, **axpy**, **copyVec** и **fillConstVal**, которые принимают в себя помимо основных аргументов параметр для измерения времени.

Процедуры **spmvCount**, **dotVecCount** и **axpyCount** являются вспомогательными и нужны для подсчета числа операций с плавающей точкой для соответствующих функций.

В функции **generate** производится обработка входных параметров задачи и формирование **StartMatrix** и **ELLPACK**.

В **fill** – заполнение матрицы значений A из ELLPACK соответствующим методом и заполнение вектора правой части b .

В **solve** – решение задачи методом сопряженных градиентов.

В **result** – вывод результатов измерений.

Для запуска программы необходимы 6 аргументов: N_x , N_y , K_1 , K_2 , `maxit`, `eps`. Седьмой параметр `Y` опциональный. При его вводе будут показаны результаты, полученные в задаче, такие как стартовая матрица, ее представление в формате ELLPACK, матрица A , вектор b и x .

3 Исследование производительности

3.1 Характеристики вычислительной системы

Запуск задания производился на Polus со следующими характеристиками:

Пиковая производительность	55.84 TFlop/s
Производительность (Linpack)	40.39 TFlop/s
Вычислительных узлов	5

На каждом узле:

Процессоры IBM Power 8	2
NVIDIA Tesla P100	2
Число процессорных ядер	20
Число потоков на ядро	8
Оперативная память	256 Гбайт

Коммуникационная сеть	Infiniband / 100 Gb
Система хранения данных	GPFS
Операционная система	Linux Red Hat 7.5

Память:

На одном узле — 2 сокета

На каждом сокете:

4 планок DDR-2400 по 32 GB (Всего 128 GB)

2 кентавра (L4 кэш + контроллер памяти)

Кентавр - 4 канала памяти

(8 каналов на сокет, 16 каналов на узел)

L1 кэш – 64KB

L2 кэш – 512KB

L3 кэш – 8 MB/ядро

L4 кэш – 16 MB на кентавр (32 MB на сокет)

Пиковая пропускная способность на сокете – 153.6 GB/s

Пиковая пропускная способность на узле – 307.2 GB/s

Компиляция программы производилась следующим образом:

g++ -O3 -fopenmp prас.cpp -o prас

g++ (GCC) 9.3.1 20200408 (Red Hat 9.3.1-2)

3.2 Результаты измерений производительности

3.2.1 Последовательная производительность

Для проверки производительности рекомендуется взять число элементов равное следующим величинам

- 1 000 000
- 10 000 000

Для начальных параметров Nx, Ny, K1 и K2 выбраны следующие значения:

- 750 1000 2 1
- 2500 3000 2 1

Параметры maxit и eps предлагается взять равными 2000 и 0.001 соответственно.

Проверка потребления памяти:

1) Вход: 750 1000 2 1 2000 0.001

- $N_{cells} = 750 * 1000 = 750\,000$
- $size = 1\,000\,000$
- $matrix: 750\,000 * 12 = 9\,000\,000$ байт = 8.583 MB
- $A : 1\,000\,000 * 5 * 8 = 40\,000\,000$ байт = 38.147 MB
- $JA: 1\,000\,000 * 5 * 4 = 20\,000\,000$ байт = 19.073 MB
- $currentPos: 1\,000\,000 * 4 = 4\,000\,000$ байт = 3.815 MB
- Пиковое (генерация) = 73 000 000 байт = 69.618 MB
- $b: 1\,000\,000 * 8 = 8\,000\,000$ байт = 7.629 MB
- Пиковое (заполнение) = 68 000 000 байт = 64.850 MB

2) Вход: 2500 3000 2 1 2000 0.001

- $N_{cells} = 2\,500 * 3\,000 = 7\,500\,000$
- $size = 10\,000\,000$
- $matrix: 7\,500\,000 * 12 = 90\,000\,000$ байт = 85.831 MB
- $A : 10\,000\,000 * 5 * 8 = 400\,000\,000$ байт = 381.470 MB

- JA: $10\,000\,000 * 5 * 4 = 200\,000\,000$ байт = 190.735 MB
- currentPos: $10\,000\,000 * 4 = 40\,000\,000$ байт = 38.147 MB
- Пиковое (генерация) = 730 000 000 байт = 696.182 MB
- b: $10\,000\,000 * 8 = 80\,000\,000$ байт = 76.294 MB
- Пиковое (заполнение) = 680 000 000 байт = 648.499 MB

Как мы видим, потребление памяти растет линейно относительно размера задачи.

Для вычисления GFLOPS использовалась формула:

$$\text{GFLOPS} = \frac{F}{T \times 10^9}$$

F – число операций с плавающей точкой;

T – время работы.

Значение F считалось отдельно для каждой из функций `spmv`, `spmvDiag`, `dot`, `axpy`. В функции `solve` операция `spmv` применяется один раз, `spmvDiag` - один раз, `dot` – дважды, `axpy` – трижды.

Общее число операций находится путем умножения полученных от вспомогательных функций **`spmvCount`**, **`spmvDiagCount`**, **`dotVecCount`** и **`axpyCount`** значений на число операций цикла `n` и количество использований в алгоритме.

При выполнении программы на одном потоке мы получаем следующие значения GFLOPS для N равного 1 000 000 и 10 000 000 соответственно:

- | | | |
|---------------------------|------|------|
| • <code>spmv</code> : | 1.99 | 2.01 |
| • <code>spmvDiag</code> : | 0.42 | 0.40 |
| • <code>dotVec</code> : | 1.25 | 1.31 |
| • <code>axpy</code> : | 1.92 | 2.30 |

Исследование зависимости достигаемой производительности от размера системы N проводится в пункте 3.2.2 Параллельная производительность. Время выполнения последовательной версии считается равной времени выполнения программы на одном потоке.

3.2.2 Параллельная производительность

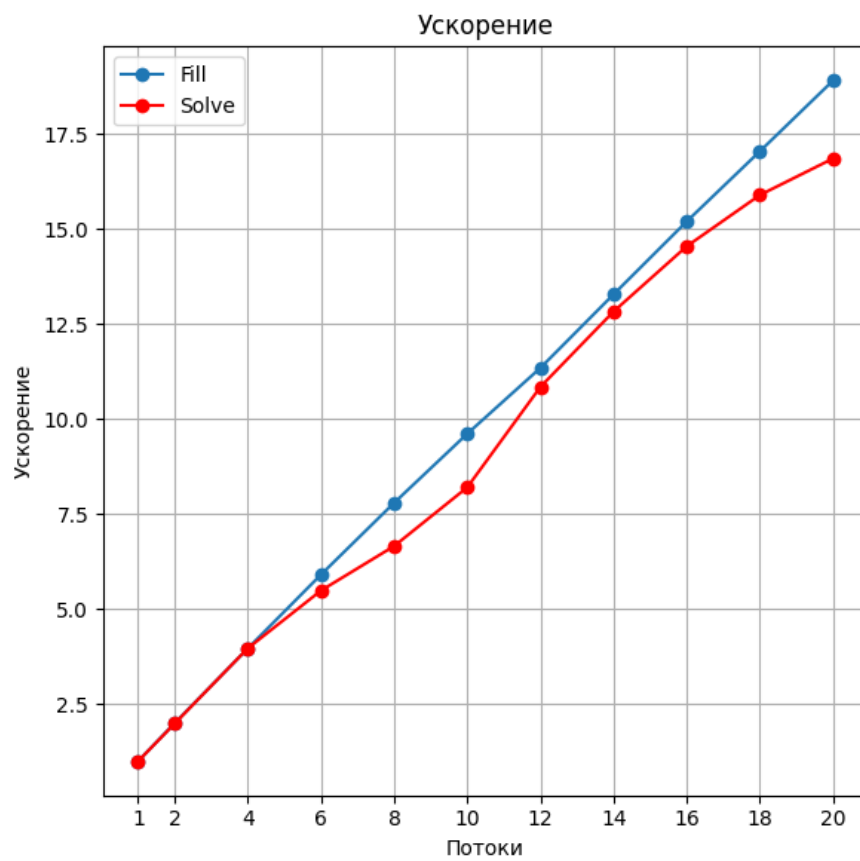
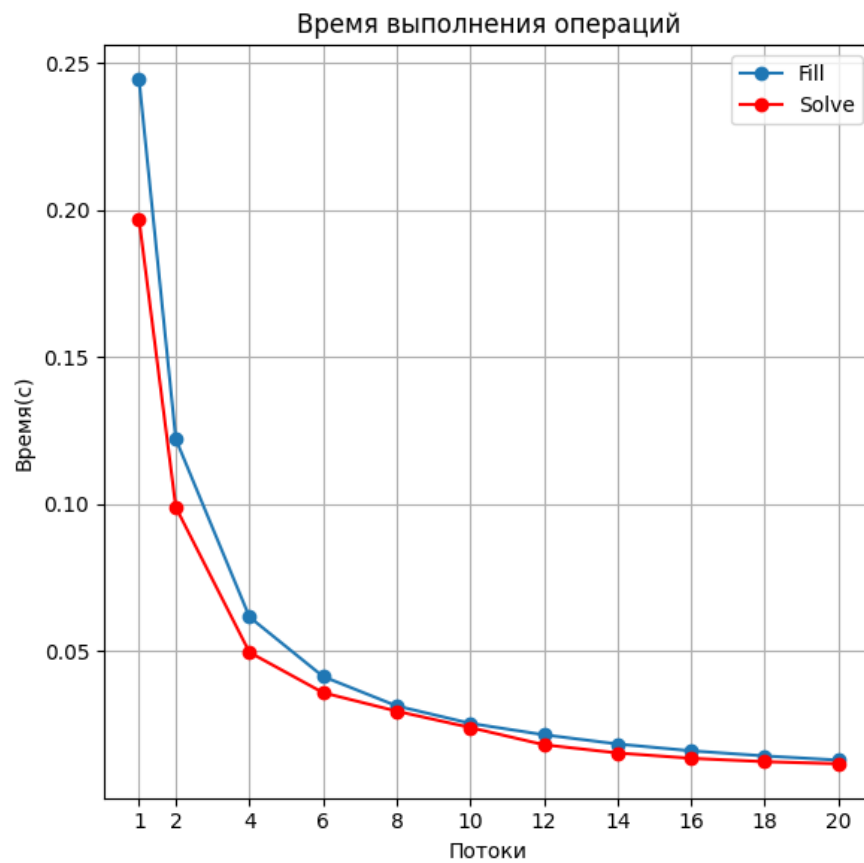
Ниже приведены графики, иллюстрирующие следующие данные для каждого N:

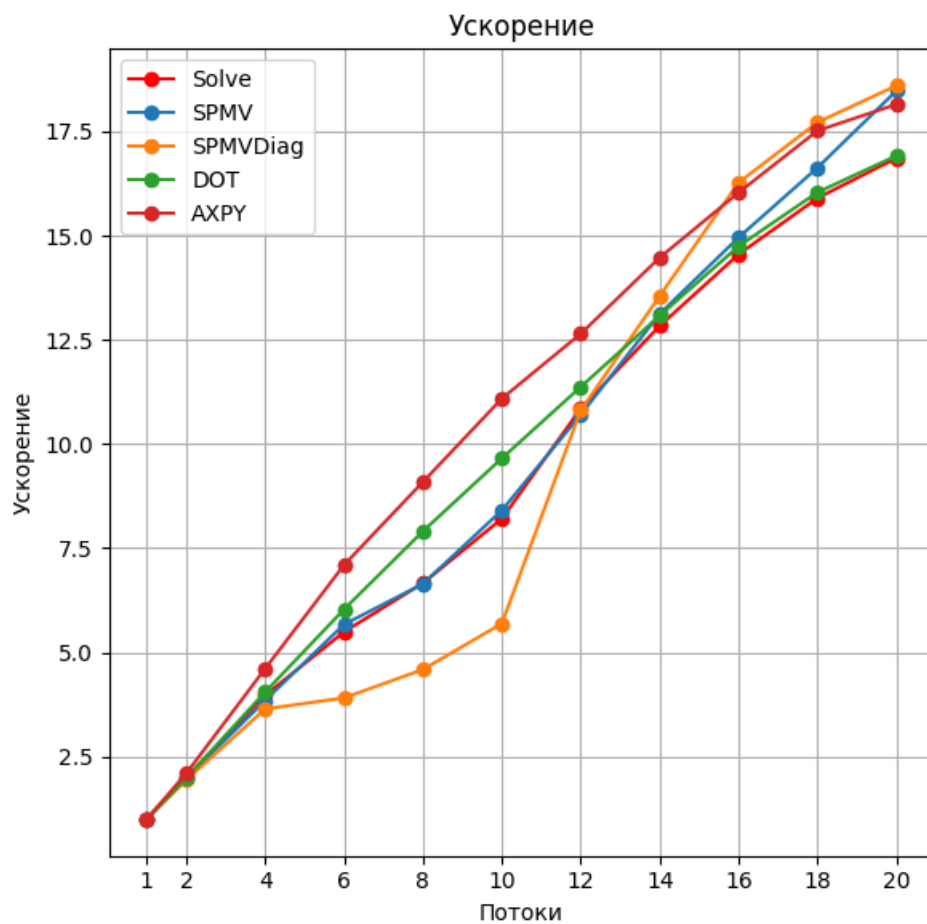
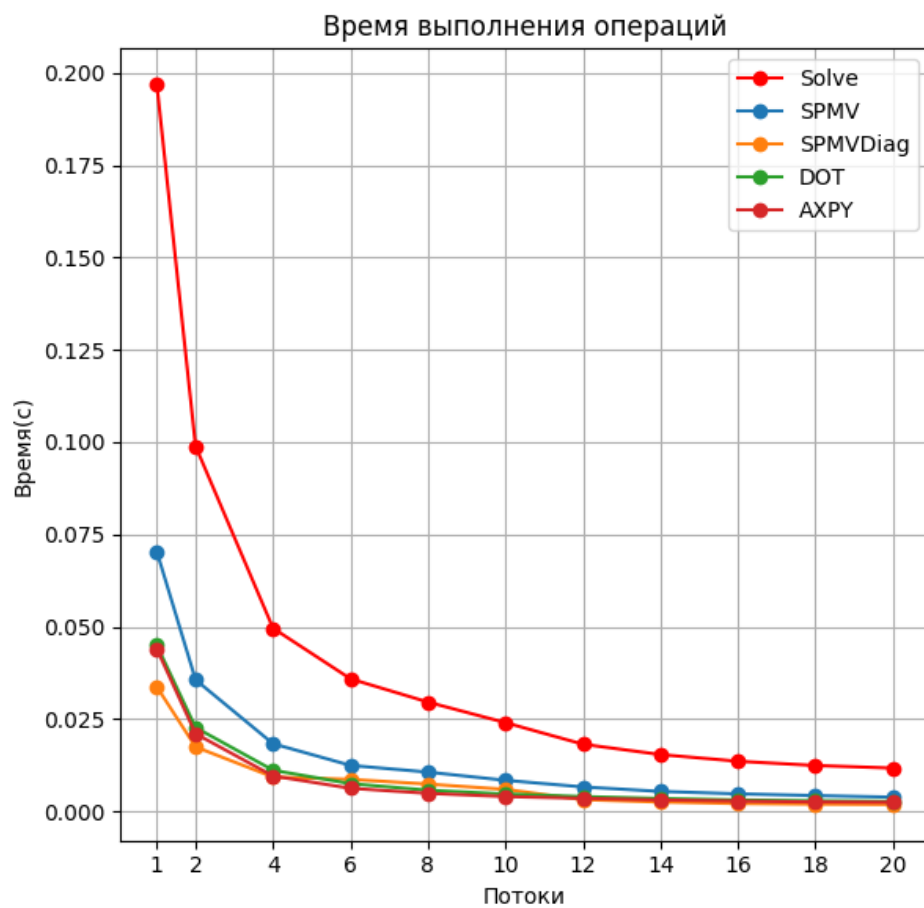
- Зависимость времени выполнения основных этапов программы от числа потоков
- Зависимость ускорения основных этапов программы от числа потоков
- Зависимость времени выполнения этапа Solve и трех базовых операций программы от числа потоков
- Зависимость ускорения этапа Solve и трех базовых операций программы от числа потоков
- Зависимость GFLOPS этапа Solve и трех базовых операций программы от числа потоков
- Зависимость получаемого процента эффективности этапа Solve и трех базовых операций программы от числа потоков

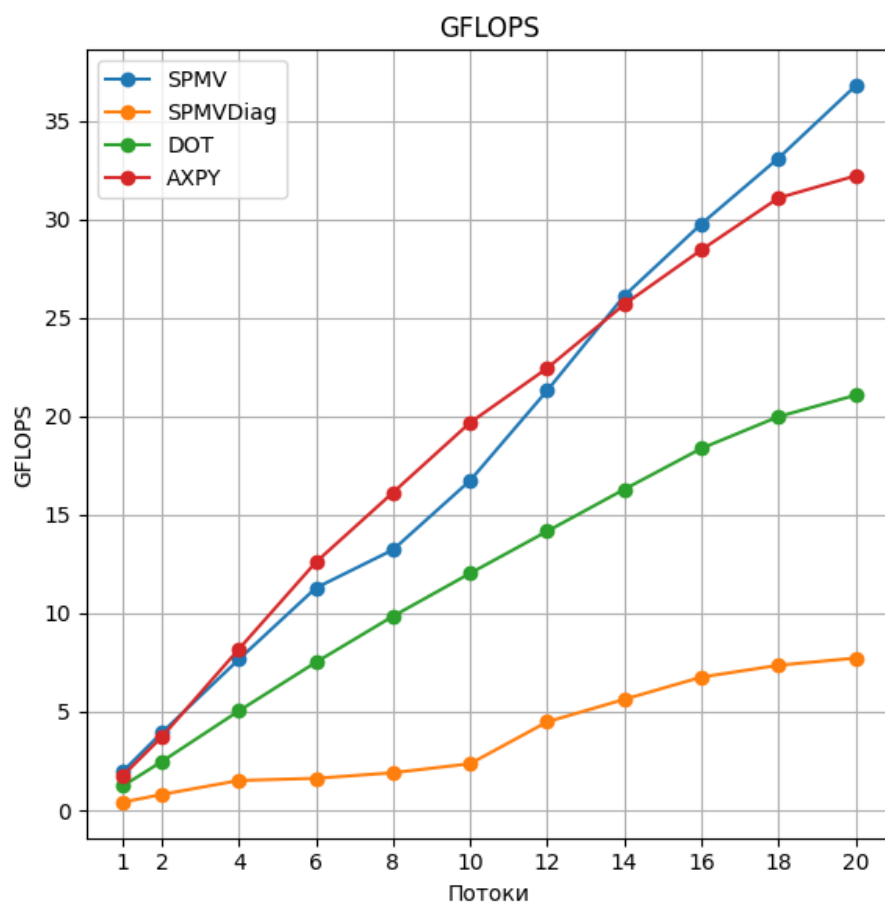
Стоит отметить, что время работы этапа Generation мало по сравнению с Fill и Solve, потому оно будет приведено в таблице вместе с его ускорением, низкое значение которого объясняется последовательным выполнением некоторых функций названного этапа.

1) N = 1 000 000

Threads	Generation Time	Speedup
1	0.0231606	1.000000
2	0.0164673	1.406916
4	0.0141040	1.642528
6	0.0136398	1.698113
8	0.0136025	1.702732
10	0.0137668	1.682092
12	0.0137817	1.680276
14	0.0140232	1.651507
16	0.0140312	1.650566
18	0.0142927	1.618998
20	0.0142965	1.618566

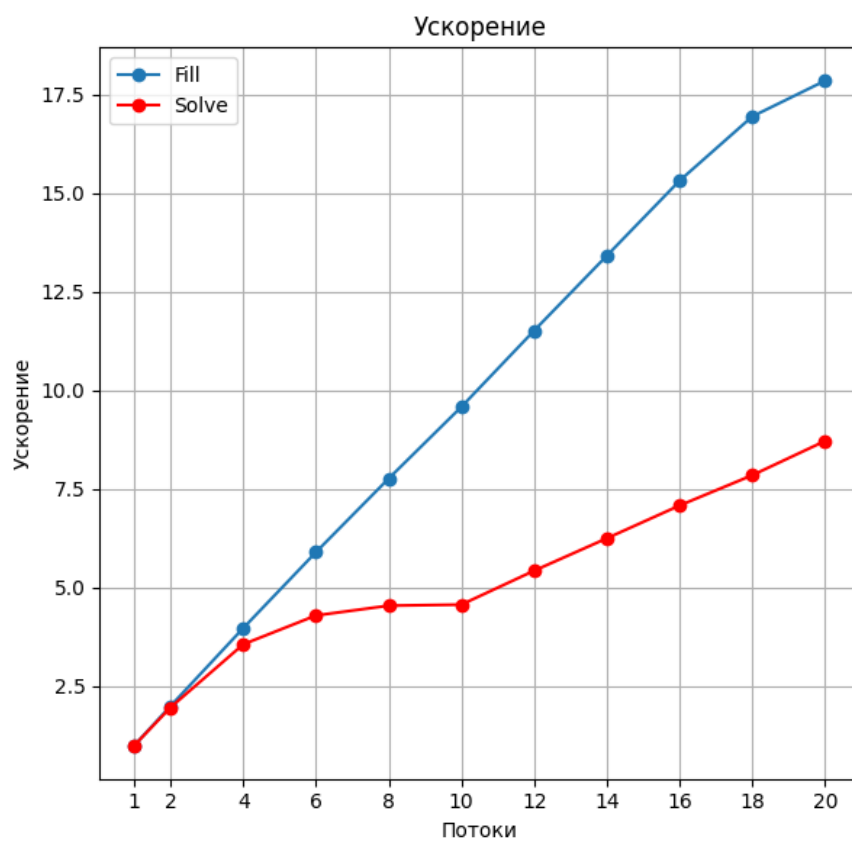
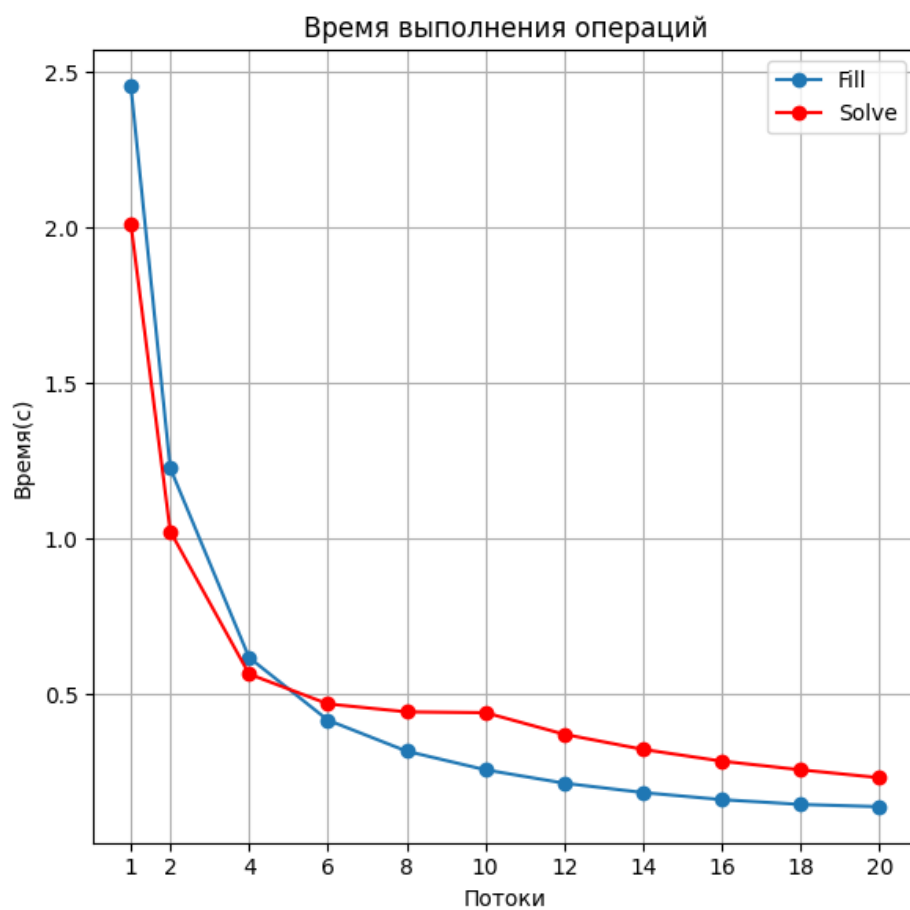


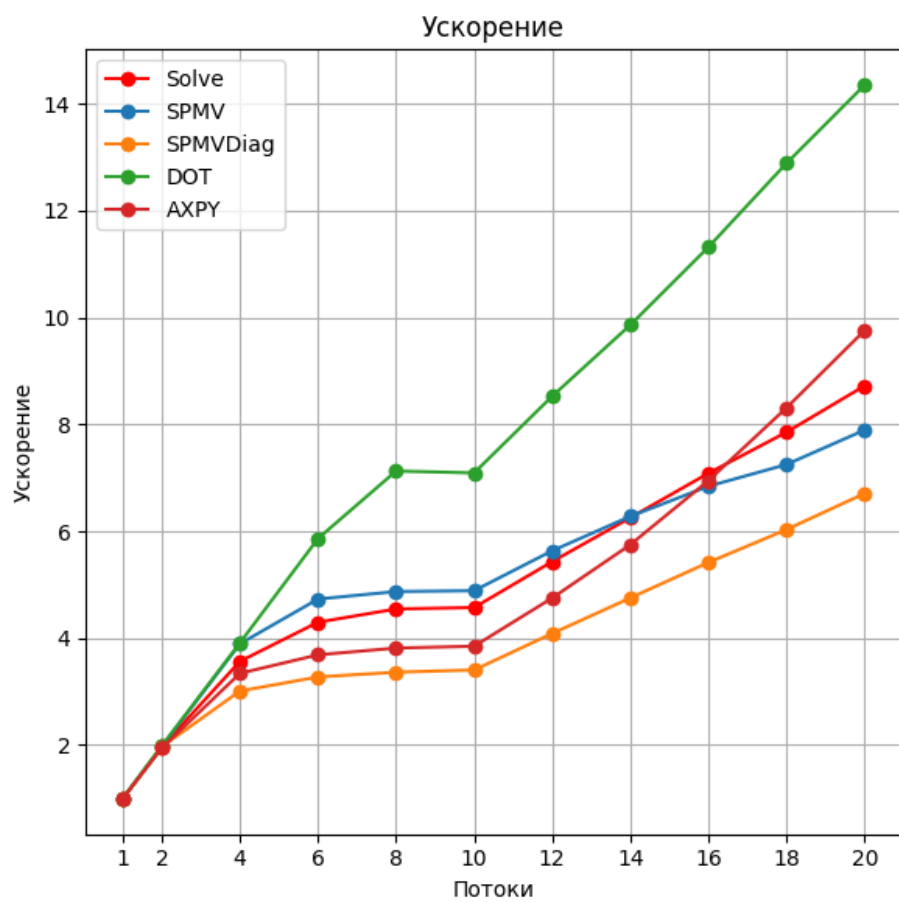
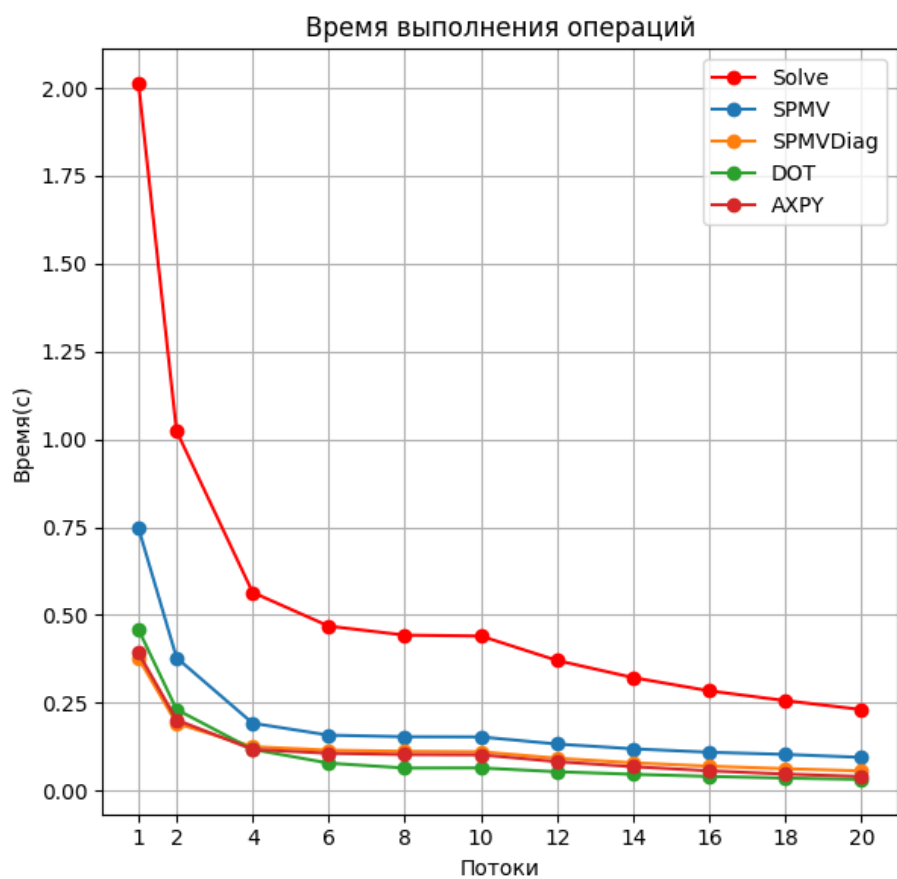


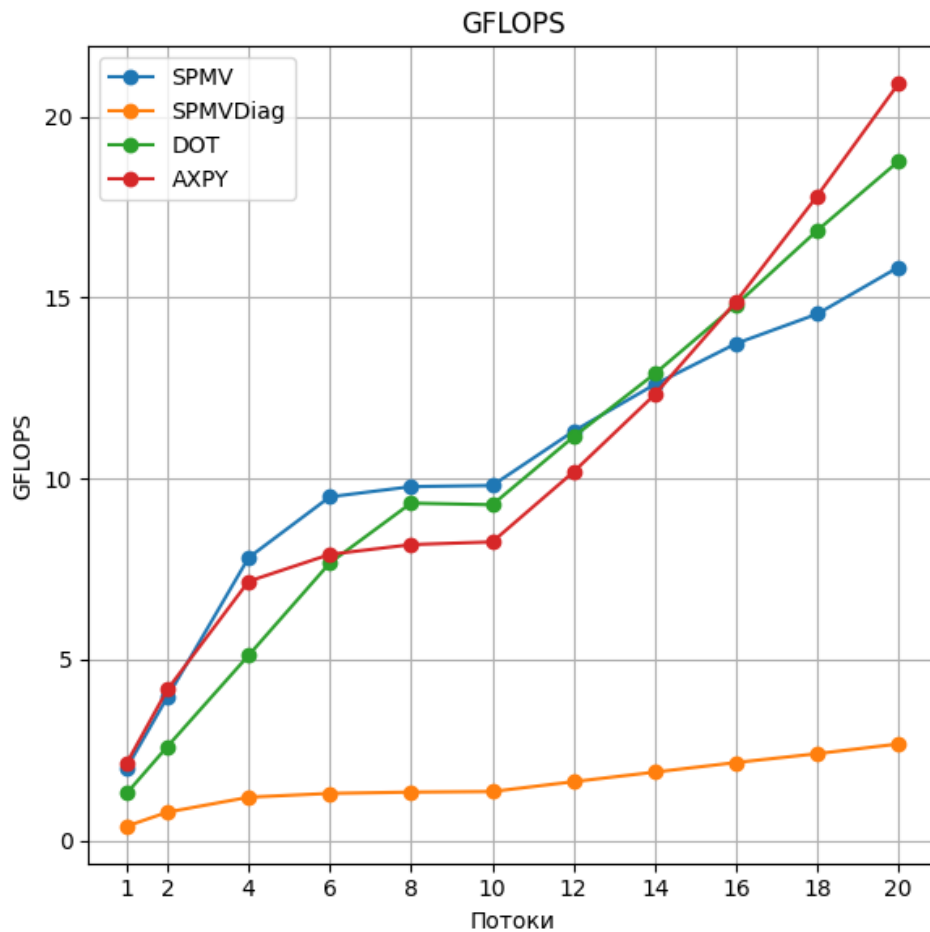


2) $N = 10\,000\,000$

Threads	Generation Time	Speedup
1	0.231967	1.000000
2	0.166260	1.394642
4	0.133059	1.743232
6	0.124071	1.869841
8	0.121127	1.916124
10	0.119937	1.935889
12	0.118180	1.962413
14	0.116391	1.993967
16	0.115200	2.013951
18	0.114645	2.022220
20	0.114304	2.028476







4 Анализ полученных результатов

Для определения эффективности операций необходимо посчитать TBP, что вычисляется согласно формуле:

$$TBP = \min(TPP, BW * AI)$$

TPP - пиковая производительность устройства

BW – пропускная способность памяти – GB/s

AI – вычислительная интенсивность – FLOP/byte

Напомним, как выглядят исследуемые операции, и посчитаем для них операционную интенсивность:

MAX_ELEMS_IN_ROWS = 5

```

void spmv(double *qk, const ELLPACK &ell, const double *pk) {
    #pragma omp parallel for
    for(int i = 0; i < ell.size; i++) {
        double sum = 0;
        for(int k = 0; k < MAX_ELEMS_IN_ROWS; k++) {
            sum += ell.A[i*MAX_ELEMS_IN_ROWS+k] * pk[ell.JA[i*MAX_ELEMS_IN_ROWS+k]];
        }
        qk[i] = sum;
    }
}

void spmvDiag(double *zk, const ELLPACK& ell, const double *rPred, const int size) {
    #pragma omp parallel for
    for (int i = 0; i < size; i++) {
        zk[i] = rPred[i]/ell.A[i*MAX_ELEMS_IN_ROWS];
    }
}

double dotVec(const double* x, const double* y, const int size) {
    double res = 0;
    #pragma omp parallel for reduction(+:res)
    for (int i = 0; i < size; i++)
        res += x[i]*y[i];
    return res;
}

void axpy(double* pkVec, const double* zk, const double bk , const double* pkVecPred, const int size) {
    #pragma omp parallel for
    for (int i = 0; i < size; i++)
        pkVec[i] = zk[i] + bk*pkVecPred[i];
}

```

Для `spmv` на каждую строку имеем одно сложение и умножение. Итого $2 \times 5 = 10$. При этом мы делаем 3 обращения к памяти: `ell.A[..]` – double 8 байт, `pk[..]` – double 8 байт, `ell.JA[..]` – int 4 байта. Итого $5 \times (8+8+4) = 100$ и запись в `qk[i]` – double 8 байт $\Rightarrow$ 108 байт. Значит имеем $10/108 = 0.0926$ FLOP/байт.

Посчитав точно также операционную интенсивность остальных операций, получим:

- `spmv` – 0.0926 FLOP/байт
- `spmvDiag` – $1/(8+8+8) = 0.04167$ FLOP/байт
- `dotVec` – $2/(8+8) = 0.125$ FLOP/байт
- `axpy` – $2/(8+8+8) = 0.08333$ FLOP/байт

Пиковая пропускная способность на узле – 307.2 GB/s.

Получаем следующие ТВР для исследуемых операций:

- `spmv`: 28.4
- `spmvDiag`: 12.8
- `dotVec`: 38.4
- `axpy`: 25.6

Gflops на 20 потоках для 1 000 000 и 10 000 000 соответственно:

•	smpv:	36.8	15.9
•	spmvDiag:	7.8	2.7
•	dotVec:	21.1	18.8
•	axpy:	32.3	21.0

Тогда эффективность будет равна:

•	smpv:	129.6%	56%
•	spmvDiag:	61%	21.1%
•	dotVec:	55%	49%
•	axpy:	126.2%	82%

По результатам видно, что при $N = 1 \cdot 10^6$ эффективность выше чем при $N = 1 \cdot 10^7$. В отдельных случаях эффективность даже превышает 100%, что объясняется кэшированием данных.

Эффективность операции smpv при $N = 1 \cdot 10^6$ выше 100%, но при $N = 1 \cdot 10^7$ она падает до ожидаемых 56%.

spmvDiag является операцией деления и имеет низкую операционную эффективность, что приводит к низкой эффективности.

dotVec показывает эффективность в 50 %, что является ожидаемым результатом и объясняется наличием операции reduction.

axpy достигает наивысшей эффективности в 126 % и 82% при N равном $1 \cdot 10^6$ и $1 \cdot 10^7$ соответственно за счет хорошей векторизации и отсутствию операций синхронизаций.